Computational Thinking with Python — Part II

Class 2 — Representation, Data, State

Class goal

Today we continue focusing on **State**, but we begin from a concrete design problem rather than a formal definition. At the end of the previous class, we wrote a function that lets us buy a stock. The code looks reasonable at first. But if we think more deeply about what the function is actually supposed to represent, the model begins to crack.

The larger goal here is to build two habits:

Habit	Why it matters
Think deeply about repeated state changes	A function that seems fine on one call may fail once it is called again.
Account for every case through time	When modeling a dynamic system, every case must be handled exactly once.

Where we left off

At the end of the previous class, we had a portfolio structure and a function that buys stock. This is our starting point today.

```
port = {'cash':10000.00,
        'positions' : [{'sym':'AAPL',
                        'shares':10,
                        'price':190.00},
                       {'sym':'MSFT',
                        'shares':5,
                        'price':420.00}]}

def buy_stock(port):
    sym = input('Enter stock symbol you want to buy: ')

    txt = 'Enter shares you want to buy: '
    while True:
        try:
            shares = int(input(txt))
            break
        except ValueError:
            txt = 'You must enter an integer: '
```

Spanning the Possibility Space

Before coding, ask: **Have I considered all the ways this system can behave?**

- What states can exist?
- What transitions are possible?
- Where could things break?

Common failure modes:

- Duplicate records
- Missing data
- Out-of-order rows
- Invalid transitions
- Non-trading days appearing in data

Key question: What would have to go wrong for this system to fail?

Core idea

A representation can look fine at first and still break once the

```
    txt = f'Enter price of {sym}: '
    while True:
        try:
            price = float(input(txt))
            break
        except ValueError:
            txt = 'You must enter a float: '

    if shares * price > port['cash']:
        print('insufficent cash. transaction denied')
        return None
    else:
        new_position = {'sym': sym,
                        'shares':shares,
                        'price':price}

        port['positions'].append(new_position)
        return None
```

system evolves through time.

Broader point

Good programming involves understanding why representations are designed the way they are.

Exercise 2.1 — What exactly happens if the same stock is bought twice?

Suppose the portfolio already contains AAPL and then `buy_stock(port)` is called and more AAPL is bought.

1. Should the new purchase overwrite the old one?
2. Should the function append a second AAPL entry?
3. Should the two holdings be combined?
4. What information might be lost under each choice?

Big idea: the issue is not just whether the code runs. The deeper issue is whether we have thought carefully enough about what state the function is changing, what repeated calls mean, and what the system is supposed to remember.

Thinking deeply before coding more

When we design or examine a function, we have to think deeply about exactly what we want to accomplish. We must think carefully about all the possibilities for how the state could change, especially when the function is called repeatedly. A shallow solution may look correct on the first call and still fail as a model of the system.

This leads to a broader lesson. More generally, understanding **why** things are done the way they are is critical to success in programming. Most recurring patterns are not accidents.

The same idea applies not just to the code we write, but to the language itself. When something appears repeatedly in Python, it is worth asking: is this a coincidence, or is there a real design reason behind it?

Consider the Python language. Look at the `range` function and slicing. What exactly is Python doing, and why?

Class 2 — A recurring Python pattern

Use a familiar Python pattern to see how a representation choice can make programming easier.

Where has this pattern appeared already?

- `range(start, end)`
- Slices such as `text[start:end]`

Both behave the same way: the start is included, and the end is excluded.

Exercise 2.2 — Work concretely first, then summarize

Below are two ways to represent adjacent intervals.

```
Inclusive / Exclusive:
[0,5) and [5,10)
actual values: [0,1,2,3,4] and [5,6,7,8,9]

Inclusive / Inclusive:
[0,5] and [5,10]
actual values: [0,1,2,3,4,5] and [5,6,7,8,9,10]
```

1. Using the values shown above, concatenate the two lists under each convention.
2. Use what you observe to complete the table below.

Property	`[0,5) + [5,10)`	`[0,5] + [5,10]`
Length formula		
Concatenation result		
Boundary handling		
Gap check		

After completing the table, answer this question: Which convention seems easier to program with, and why?

Pattern to notice

`range` and slicing follow the same rule. That is a clue that the rule is probably useful.

What to watch for

Compare what happens at the boundary value. Does the representation handle the boundary automatically, or does extra logic have to be added?

Looking ahead

This same idea will matter later when representing holdings through time.

Inclusive start, exclusive end

At first, this may look like a small Python convention. But it is really an example of a deliberate representation choice.

Five reasons this pattern is so useful

Reason	Why it helps
Length is simple	The size is simply `end - start`, with no extra `+1`.
Concatenation is clean	Adjacent intervals fit together naturally without duplicate boundary values.
Gap checking is simple	Two intervals meet cleanly when the first interval's end equals the second interval's start.
No gaps, no duplicates	Each value belongs to exactly one interval rather than being missed or counted twice.
Natural for state changes	A new state can begin exactly at the boundary where the old one stops.

Exercise 2.3 — Apply the same idea to time

Suppose one interval runs from Jan 15 to Feb 10 and the next begins on Feb 10.

1. What ambiguity appears if both intervals include Feb 10?
2. How does inclusive start, exclusive end avoid that problem?

Key idea: good representations cover the possibilities exactly once. This Python pattern is a reminder that useful representations are chosen for reasons. The same standard should be used when modeling systems through time.

Class 2 — Returning to the portfolio problem

Now apply the same standard of careful representation to the portfolio system.

The current model is still too shallow

The `buy_stock` function exposed one problem: repeated purchases of the same stock are not represented clearly. But the deeper problem is larger. The current design may let us store a portfolio at one moment, yet it does not let us answer many natural questions about the portfolio **through time**.

Exercise 2.4 — What questions should a real portfolio system answer?

Write down at least four questions that a useful portfolio system should be able to answer.

Examples of the kind of questions to think about:

Important shift: once time matters, it is no longer enough to store only the current position. We need a way to record the changes that happened over time.

From positions to transactions

If the portfolio changes over time, one natural idea is to store the **transactions** rather than only the current holdings. A transaction is a dated event that changes the state of the portfolio.

Core move

Instead of storing only "what the portfolio is now," store the dated events that caused the portfolio to become what it is now.

Notice the connection

The portfolio is a dynamic system. Once time matters, the representation must account for every state change through time.

Representation reminder

A static snapshot may hide the very history needed to answer the most important questions.

Naming convention reminder

When field names begin to multiply, aim for names that are easy to understand, short, and used consistently across similar fields.

For example, a name like `trn_type` makes the role of the field

Exercise 2.5 — What belongs in a transactions file?

Suppose a transactions file is the primary record of what happened. What transaction types would be needed for a simple equity portfolio?

For each transaction, what fields should be stored?

explicit. Short coded values such as `CNTRB` and `WDRW` may also be useful if they are documented clearly and used consistently.

A first transaction structure

A reasonable first attempt might look like this:

Field	Possible meaning
date	When the transaction occurred
trnType	contribution, withdrawal, buy, sell
ticker	The stock involved, if any
shares	Number of shares, if applicable
price	Transaction price per share, if applicable
cashAmount	Cash amount for a contribution or withdrawal

Exercise 2.6 — Transactions as system history

If transactions are stored through time, what current state and historical information should the system be able to reconstruct from that history?

1.
2.
3.
4.

What functions should this transaction system support?

Once transactions become the main stored history, the next question is what functions and reports the system should be able to build from them.

Function	Purpose
`create_transaction(...)`	Create one of four transaction types: contribution, withdrawal, buy, or sell
`get_cash_balance(as_of_date)`	Reconstruct the cash balance on a chosen day
`build_portfolio(as_of_date)`	Reconstruct the portfolio on a chosen day from transaction history
`list_transactions_for_ticker(ticker)`	Show the dated transaction history for one ticker

Design choice: treat cash as ticker `$$$$`. Its price is always `1.00` and it is always valid. When a stock transaction is created, the matching cash transaction should also be added so the transaction history stays balanced.

A first portfolio report

A portfolio report built from transaction history might include:

Field	Meaning
shares	Current shares held
cost	Cost basis of the position
market price	Price on the chosen valuation date
realized gain/loss	Gain or loss on shares that have already been sold
unrealized gain/loss	Gain or loss on shares still being held
total contributions	Total cash contributed to the portfolio
total withdrawals	Total cash withdrawn from the portfolio

If a position goes to zero shares, it should still appear in a historical report so realized gains or losses are not lost.

Exercise 2.8 — What would the portfolio look like after a partial sale?

Suppose the portfolio begins with a contribution of `$100,000`. Then:

1. Buy `500` shares of `XYZ` at `$100` per share.
2. Assume the price rises by `$1` each day.
3. After `10` days, sell `100` shares.

Assume no commissions and use FIFO. Build the portfolio report on the day of the sale. Show:

- remaining shares
- remaining cost basis
- cash balance
- market price
- market value
- realized gain/loss
- unrealized gain/loss
- total portfolio value

Extension: What should this report look like if all `500` shares were sold instead? Should `XYZ` still appear in the report?

Exercise 2.7 — Use AI to help build one report

Choose one target report: cash balance on a given day, portfolio on a given day, or transaction history for one ticker.

1. First describe in plain English exactly what the function should return.
2. Then ask AI to write a first draft of the code.
3. Inspect the code carefully. What assumptions did AI make about the transaction file?
4. What test cases would you use to decide whether the code is correct?

Class 2 — From transactions to valuation through time

Once transactions are stored, the next natural question is what the portfolio is worth on any date.

What else should a portfolio system do?

If transactions let us reconstruct the holdings, then a natural next question is:

What is the portfolio worth on any given date?

That question introduces a new requirement. It is not enough to know how many shares are held. We also need a way to find the relevant prices on the date being examined.

Exercise 2.8 — What is needed to value the portfolio through time?

List the kinds of information the system would need in order to compute portfolio value on any trading day.

Data resources the system will need

To value the portfolio through time, the system will need more than a transactions file. It will need a small data ecosystem.

Data resource	Why it is needed
Universe of tradable tickers	So the system knows which tickers exist in the market data universe at all
Availability dates for each ticker	So the system knows the start and end dates during which each ticker is eligible to appear in that universe
Market date file	So the system knows which dates the market is open and which dates are valid valuation dates
Pricing table	So the system can value positions on a chosen date

Same principle again

Every point in time should map to exactly one valid representation of the portfolio and exactly one relevant price record.

Important distinction

Close prices help value the portfolio. Adjusted prices are useful for return analysis. These are different purposes.

Looking ahead

Once holdings and prices are both available through time, the system can compute value through time.

Dividends	So the system can model cash distributions or total return behavior correctly
Splits	So the system can adjust share counts correctly
Spin-offs and other corporate actions	So the system can model value leaving one asset and appearing in another

Design note: one practical choice is to store dates as `YYYYMMDD`. That representation sorts naturally and can sometimes simplify early programs because many comparisons can be done directly without importing date objects right away.

Exercise 2.10 — Build a ticker availability file

Create a `sp100_tickers.json` file that defines when each ticker is tradable using half-open intervals `[start, end)`.

Use these conventions:

- dates must follow the market date convention `YYYYMMDD`
- intervals are stored as `[start, end)`
- if a ticker is still active, use `99999999` as the end date

Suppose ticker `XYZ` trades on every market date in January, does not trade in February, and then trades again from March onward. How should that be represented?

```
{
  "XYZ": [
    {"start": "20250101", "end": "20250201"},
    {"start": "20250301", "end": "99999999"}
  ]
}
```

Why is this better than assuming every ticker has only one start date and one end date?

Exercise 2.10 — Build a market date file from the pricing data

Start by getting your hands dirty with the data. Use the pricing file to extract the unique trading dates, sort them, and save them as a JSON file.

Your program should answer questions such as:

- How many unique market dates are in the file?
- What is the first market date?
- What is the last market date?
- Does every ticker appear on every market date?

What might this file later help us validate?

Exercise 2.12 — Use AI to help build `sp100_tickers.json`

Now use AI to help write the code that builds the ticker availability file from the pricing file and the market dates file.

1. Before using AI, explain in plain English what counts as one continuous trading interval.
2. Write a prompt that asks AI to find each ticker's continuous date ranges and convert them into `[start, end)` intervals.
3. Tell AI to use `99999999` for open-ended intervals.
4. After AI writes code, inspect the boundaries carefully. Did it really implement `start <= date < end`?
5. What test data would you create to check whether gaps are handled correctly?

A pricing table through time

A pricing table is often stored one row per date. But for this course, it may be useful to think about a representation that also makes the

valid interval explicit. For example, a pricing row could include a start date and an end date for the period during which the row is valid.

Exercise 2.12 — Would interval-based price rows make programming easier?

Suppose each pricing row had:

- ticker
- startDate (inclusive)
- endDate (exclusive)
- closePrice
- adjClosePrice

What might become easier? What might become less realistic compared with how market data is usually stored?

Class 2 — Corporate actions and data interpretation

A real system must decide what different corporate actions mean for state, not just for returns.

Not all data fields mean what they first appear to mean

Once prices, dividends, and corporate actions are introduced, another important modeling question appears:

What does the data mean for the state of the portfolio?

A split, a dividend, and a spin-off may all affect value, but they do not affect the state of the portfolio in the same way.

Event	What changes in the portfolio system?
Dividend	Cash is distributed; share count usually stays the same
Split	Share count changes; price adjusts; value is preserved
Spin-off	A new asset appears; value leaves one asset and appears in another

Exercise 2.13 — Why is a spin-off not just a split?

Write a short explanation of why a spin-off should not be modeled the same way as a true stock split.

Using AI to help inspect the data and build reports

One useful role for AI is to help write small programs that test ideas about the data or build reports from transaction history. For example, if close prices are price-only and adjusted close prices reflect total return, a short program can help check whether that relationship seems to hold and whether any rows look anomalous. In the same spirit, AI can help draft functions that reconstruct cash balance, portfolio state, or ticker history from transaction records.

But before asking AI to write that code, you really have to get your hands dirty and carefully examine the structure and content of each input file. What are the columns called? What does each field mean?

Representation matters again

Yahoo-style adjusted data is useful for return analysis. A state-based portfolio system needs raw events and explicit changes in holdings and cash.

System question

Does this event change cash, shares, or the set of assets being held?

Next direction

The next major step is to define the portfolio state more formally and decide what must be stored versus what can be derived.

What is one row supposed to represent? Are dates unique within a ticker? Are duplicate rows possible? Are there missing rows? Do dividends, splits, and prices appear to be aligned the way you think they are?

That matters because if the problem is not understood, AI may write code that looks plausible and is still wrong. It may check the wrong relationship, use the wrong key, or quietly assume a data structure that does not actually exist. Understanding the data and understanding the code remain essential.

A very good way to test whether your checking code really works is to deliberately introduce errors into otherwise clean files and see whether the program catches them. You can duplicate rows, delete rows, or change values. That kind of testing is often more informative than simply running the code once on data that appears fine.

AI reminder: AI can help test an idea, but only if the idea is understood clearly enough to ask the right question, inspect the output, and recognize when the code is based on the wrong assumptions. Good prompts begin with careful examination of the actual files.

Exercise 2.14 — What should be checked in the data?

Suppose a short program is used to inspect pricing, dividends, and split information. What hypotheses should that program test?

What kinds of anomalies would be worth flagging?

As a testing strategy, suppose you intentionally modify a clean file in a few ways:

- duplicate a price row
- delete a dividend row
- change one price value
- add a duplicate dividend row

Which of these should your checking code catch directly, and which would require deeper investigation after being flagged?

Closing idea for this section: the problem is often not that the data is “bad.” The problem is that data may be designed for one purpose and then used for a different purpose. A return-oriented

representation is not always the right representation for a state-based portfolio system.